

Gestion des risques et des rendus

Projet Génie Logiciel — Suivi 2

Equipe G51

7 janvier 2026

1. Gestion des risques

Risques techniques

- **Bugs en génération de code (étape C)**
 - Impact : important
 - Mesures préventives : Tests unitaires systématiques, tests de non-régression.
 - Mesures correctives : Isolation du bug avec tests minimaux, débbug pas à pas avec `ima`.
- **Bugs Contextuelles : liaison entre les 3 passes**
 - Mesures préventives : Tests pour chaque passe et chaque regle .
 - Mesures correctives : Isolation du bug avec tests minimaux, revue de code collective.
- **Gestion de la pile machine abstraite**
 - Mesures préventives : Respect des conventions de ASM.
 - Mesures correctives : Ajout de vérifications .
- **Les Tests : comment les faire correctement ?**
 - Mesures préventives : respecter la structure des tests : valide/invalid
 - Mesures correctives : corriger l'emplacement .

Risques organisationnels

- **Retard rendu intermédiaire (12/01) ou final**
 - Mesures préventives : Planning détaillé, points d'avancement quotidiens, GitLab Issues.
 - Mesures correctives : rattrapper via les autres disciplines (Extension , Tests , Compilateur correct)
- **Mauvaise répartition des tâches**
 - Mesures préventives : Rôles définis en Agile, réunions de synchronisation , appeles Discord .
 - Mesures correctives : Ajustement des rôles.

Risques humains

- **Absence / indisponibilité**
 - Mesures préventives : Documentation partagée, binômes sur les tâches critiques, communication régulière.
 - Mesures correctives : Redistribution des tâches, rattrapage en asynchrone.
- **Conflits internes**
 - Mesures préventives : Charte d'équipe, communication bienveillante, retours constructifs.
 - Mesures correctives : Médiation, rappel des objectifs communs.

Risques outils

- **Perte de code (Git)**
 - Mesures préventives : Push régulier, branches protégées, reviews avant merge.
 - Mesures correctives : Récupération depuis dépôt distant Git, sauvegarde locale.

- **Problème configuration IDE**
 - Mesures préventives : Configuration versionnée (`.vscode/`, `.idea/`),
 - Mesures correctives : utilisation de Maven en ligne de commande.

2. Gestion des rendus

Calendrier des rendus

- **12/01/2026** : Rendu intermédiaire (sans-objet)
- **19/01/2026** : Rendu final (code, tests, docs)
- **19/01/2026** : Manuel utilisateur
- **21/01/2026** : Bilan gestion d'équipe
- **22-23/01/2026** : Soutenance

Validation avant rendu

- **Tests automatisés** : scripts `*.sh` dans `src/test/script/`
- **Couverture de code** : JaCoCo intégré à Maven.
- **Vérifications manuelles** : revue de code par un autre membre de l'équipe (condide), tests utilisateur sur exemples variés
- **Intégration continue** via la branche "master"

Gestion Git

- **Branches** :
 - **master** : version stable, livrables tagués
 - **branch-sujet** : chaque branch est dédiée à une partie ou un sujet spécifique, (par exemple "lexer" contient les ajouts liés à la vérification lexer.
- **Tags** : `rendu-intermediaire`, `partieC-ver..`, etc.

Tests de non-régression

- Exécution automatique des tests fournis (`common-tests.sh`)
- Script mine `run-all-tests.sh` qui couvre syntaxe, contexte, codegen
- Vérification que `decac -p` et `-q ...` sont idempotent sur les programmes valides
- Comparaison des sorties `.ass` avec `ima` sur une batterie de programmes de référence

Livrables d'Extension

- Recherche des algorithmes optimales (`Cordiv ...`) et Optimisation des algorithmes existants (series Taylor, Lagrange, Hermite...)
- Comparaison des sorties attendues, suivant le calcul flottant